

HONEST STATUS REPORT · NTRO PS-26151

AETHER

What Is Real, What Is Not

A plain-English status check of our dark-web threat attribution platform: what is really implemented, what is demo-only, what is still missing, what must be fixed first, and the phased plan to finish the full build.

Team VIHAR

Smart India Hackathon 2026 · Dark Web Threat Actor De-Anonymization

SEPTEMBER 2026 · READ IN 5 MINUTES

1. What Is AETHER? (One-Minute Version)

AETHER is our Smart India Hackathon tool for NTRO Problem Statement 26151: **Dark Web Threat Actor De-Anonymization**. In simple words, it helps an authorized investigator connect small clues from dark-web markets - forum posts, PGP keys, Bitcoin wallets, server fingerprints - and turn them into one clear profile of a suspect, with evidence strong enough for court. It does not hack Tor and it does not break encryption; it only correlates lawful clues the way a forensic analyst would.

The platform has three parts: a Next.js dashboard (the screen you see), a FastAPI backend (the brain, with nine analysis modules), and a database that stores every case, every piece of evidence, and a tamper-proof custody ledger. This document tells you, in plain and honest language, what is really working today, what is only a demo, what is still missing, and how we finish the full build.

73 / 74

backend tests pass (1 test uses a bad onion address)

9 modules

run in one pipeline - real math, but demo inputs today

0

real network calls to the dark web or Shodan in the pipeline

2. What We Really Built (Implemented and Working)

These features are genuine code, not mock-ups. They run on the server, store results in the database, and the dashboard shows their real output when the backend is online. Each one passed code review and, where marked, automated tests. This is the strongest part of our project and the part we should demo first.

Feature	What actually works	Watch out for
SHA-256 chain of custody	Real hash chain (like a mini-blockchain) saved in the database; a Verify button re-checks every block and catches tampering; tested with fake edits.	Works only with the backend online.
Stylometry engine	Real writing-style math (n-gram cosine similarity) comparing any two texts you paste, via API or the Stylometry Lab screen.	In the auto pipeline it compares against one hardcoded sample text.
Diurnal (sleep-pattern) engine	Real 24-hour activity math: finds the sleep trough and estimates the suspect's likely timezone from any timestamp list you give the API.	Pipeline always feeds it 54 synthetic timestamps.
Bitcoin peel-chain clustering	Real multi-input clustering algorithm exposed as an API endpoint on any transactions you post.	Pipeline always uses 2 built-in demo transactions.
Confidence scoring formula	Real calibrated formula (deterministic proofs vs AI leads, minus contradiction penalties) with full score breakdown.	Some inputs are hardcoded inside the pipeline.

Feature	What actually works	Watch out for
STIX 2.1 + CSV export	Server-side export using the official OASIS stix2 library (spec-valid by construction) and CSV with formula-injection defence.	Needs a case already stored in the database.
Case management + audit log	Create, list and store cases in PostgreSQL/SQLite; every investigator action is written to an audit log.	Single shared API key; no per-user login yet.
Security layer	API-key auth (timing-safe), rate limiting, payload limits, strict security headers, SSRF guard that rejects private/metadata targets.	Frontend embeds the default dev key.
3D knowledge graph	Three.js graph renders the real graph JSON the backend computes for a case, with Neo4j Cypher export statements.	Node labels/scores on it are partly fixed text.

3. What Is Fake or Demo (Never Claim It Is Live)

This is the part we must be honest about - with judges, with friends, with ourselves. The nine-module pipeline always finishes, but most of its **inputs are hardcoded**, so the same answers come out no matter what target you type. The code even stamps some of these rows *LIVE_SOURCE* when no network was touched. That label is wrong today, and fixing it is our top honesty task.

Piece	Why it is demo today	Risk if we call it real
Origin IP unmasking	Always answers 185.220.101.42, Munich, AS9009 - written in the code, no probe happens.	Judge types a new onion, sees the same IP, trust is gone.
Favicon mmh3 match	Hash value -129482710 is fixed; the real hash it computes is ignored.	Same as above - one fake number poisons the whole report.
JARM TLS fingerprint	The matcher is real, but the fingerprint fed to it is hardcoded.	Looks live on screen; it never scanned a server.
Logo / image dHash	Two fixed hash strings are compared; the hash function itself is a toy byte sampler.	Claims visual proof that does not exist.
PGP forum reuse	Key format check is real, but the note says reused on 3 forums - that list is fabricated.	An investigator could repeat an unverified claim.
Offline fallback dossier	If the backend is down, the UI invents a complete 94.8% case (marked not-live in code, but the screen still looks like a win).	Demo without backend shows fabricated evidence.
Search box + Engine Config	Search only shows a toast message; config Save changes nothing on the server.	Buttons that do nothing undermine trust.

One more important detail: when the backend is offline, the dashboard still shows a full case with 94.8% confidence. The code marks it as not-live, and a small OFFLINE badge appears, but the numbers look exactly as impressive as real ones. Our own project rule says: show **SOURCE UNAVAILABLE**, never fake data. We currently break that rule in the fallback path, and it must be fixed before any public demo.

4. What Is Missing Completely

These items do not exist in the repository at all. Some were promised in early slides and honestly listed as Pending in our README; others are gaps we found during this audit. None of them can be faked in a good demo, so they define the real work ahead.

Missing item	Why it matters for PS-26151
Tor / SOCKS5 crawler	Without it the platform never actually visits an authorized .onion target, so every collection step stays synthetic.
Shodan / Censys wired into the pipeline	A working API client exists, but the nine modules never call it; real provenance tags (LIVE vs DEMO) need this connection.
Neo4j graph database	We only generate Cypher text; cross-case queries and big-graph analysis need the real database.
Elasticsearch search	Planned as Stage B3; without it there is no fast search over stored evidence.
Trained stylometry model	We use n-gram math instead of the promised neural model (documented decision); a small trained model is still pending.
Court / statutory PDF report	The docs promise a sealed evidence PDF; today only STIX and CSV export exist.
User accounts and roles (RBAC)	One shared API key for everyone; a real agency needs named investigators and permissions.
Stored-result retrieval	Opening an old case silently re-runs the whole pipeline instead of reading stored results from the database.

5. What Is Critical (Fix These First)

Five problems are small in code size but huge in impact. Four protect our honesty rules; one is a hidden bug that will crash the production database path during a demo. All five can be fixed in a few days, and nothing else should start before they are done.

#	Fix	Why it comes first
1	Remove fake LIVE_SOURCE labels; label every hardcoded module DEMO_DATA.	Our core promise is truthful provenance. A wrong label in court or before a judge is fatal.
2	Stop the offline fallback from inventing a 94.8% case; show a clear DEMO MODE banner or empty state.	The most impressive-looking screen is currently the least honest one.
3	Fix the audit-log bug: evidence ID string is passed where the case number (integer) belongs.	Works in tests on SQLite, will crash on the production PostgreSQL docker setup.
4	Make case view read stored results from the database instead of re-running the pipeline.	Evidence must stay stable; re-running can change scores after the fact.
5	Repair the one failing test (bad onion address in test data).	Green tests are our proof of quality; keep the suite at 74/74.

6. How We Complete the Full Build

The plan below turns the current honest demo into the full platform the problem statement asks for. Phases are ordered so that trust is fixed first, then real data collection, then scale, then final deliverables. Each phase ends with something we can actually show.

Phase	Time	Work	Result we can show
1. Honesty pass	2-3 days	The five critical fixes above; wire user-provided text, timestamps and transactions through the whole pipeline.	Every number on screen is either real or clearly labelled DEMO.
2. Real collection	1-2 weeks	Call Shodan/Censys with API keys inside the pipeline; add Tor SOCKS5 fetch of favicon and pages for authorized targets; compute mmh3 and JARM from live responses.	Provenance tags flip to LIVE_SOURCE truthfully.
3. Memory and scale	1-2 weeks	Store and reload investigations; build graph from stored correlations; optional Neo4j import; Elasticsearch index and search.	Old cases open instantly; cross-case search works.
4. Final package	1 week	Sealed court-style PDF report; named investigator accounts; docker-compose deploy check; rehearsed demo script with the labelled benchmark case.	A complete, deployable, honest SIH submission.

What we should **not** build: a fake live-dark-web crawler that touches unauthorized targets, a paid VPN-free scraping shortcut, or any button that pretends to hack Tor. They add legal risk, not points. The problem statement rewards lawful correlation and explainable confidence, and that is exactly what phases 1-4 deliver.

7. The Honest One-Line Pitch

AETHER already has a real forensic backbone - a tamper-proof custody chain, four genuine analysis engines, standards-based STIX 2.1 export, and a hardened API - demonstrated on clearly-labelled benchmark data. What remains is honest labelling, real data sources, and persistence polish. We are roughly 60% of the way to the full build, and we know exactly where every gap is.

Strong enough to trust - honest about what is still demo.